Sorting

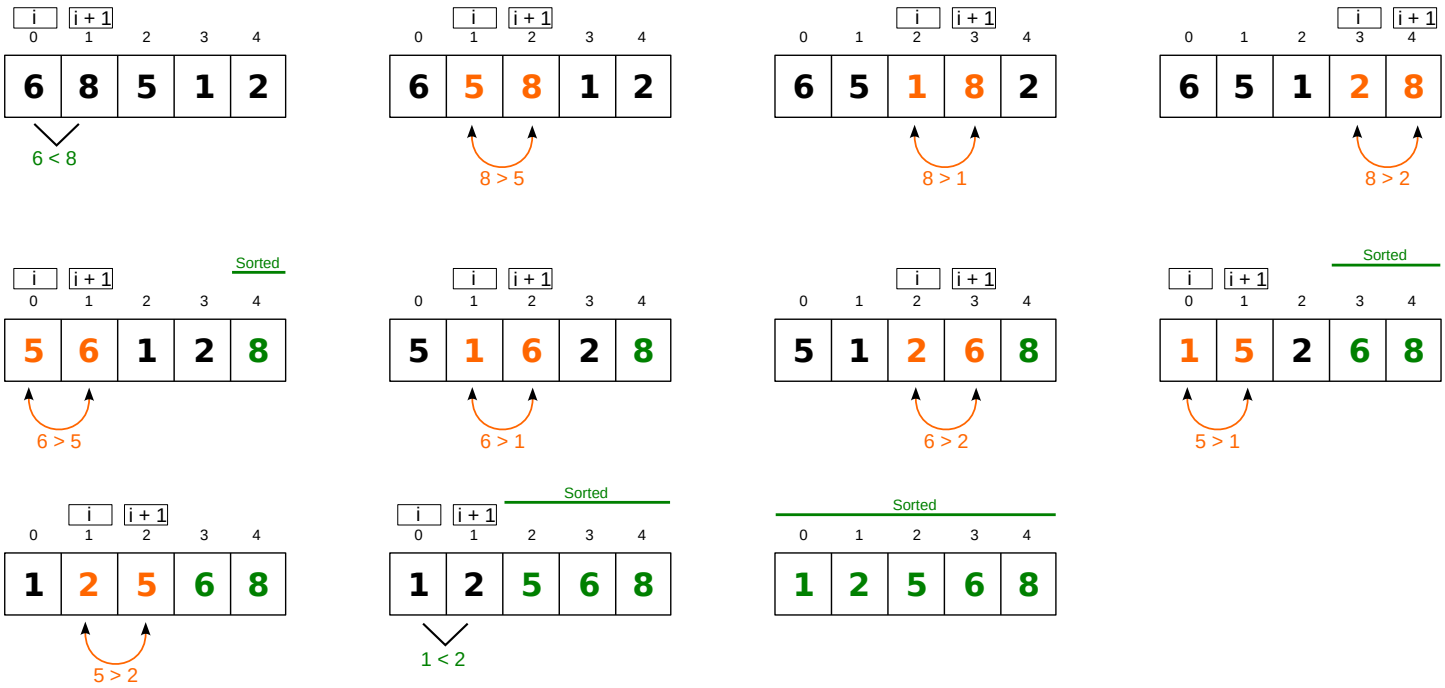
Sorting is a classical problem that has been studied for a long time. There are many cases where the solution of a problem needs to sort something to achieve the goal. In many cases though, sorting is not the goal of the problem but a step to solve it. One example can be searching for values in a list/array. The most efficient algorithms require that the array is sorted beforehand. For finding just one value, probably sorting first would be an overkill, but if the algorithm is going to need to perform several searches, then it definitively is going to benefit from a sorted array.

Sorting, as said before, is a problem in itself that has been long studied. For example it also had its role in the creation of the giant IBM. So through the years a bunch different techniques and algorithms to solve this issue have been developed. Many of them are quite easy to understand and implement, but of course, like many things in life, there are small quirks that can make the task a little more challenging.

Bubble sort

Let's start with the Bubble sort algorithm. This is probably one easiest sorting algorithms to understand. The list will be traversed repeatedly from start to end. On each step current position and the next one will be compared and swapped if necessary, so they are sorted. On each cycle, this will take the bigger value still unsorted to the end of the list (or the smaller value to the front, depending on how the array is traversed). This will be repeated until no swaps are done during the traversing, meaning that the list is sorted. The values placed at the end of the list are considered sorted and not compared anymore. That means that each cycle is a position shorter than the previous cycle.

Let's see this with an example:



In the image we can see there is a pointer `i` representing the current position. The value of the current position is compared with the one after it marked with `i+1`. The values are swapped if necessary, so the bigger value is at the right. Then `i` is moved forward 1 position and process is repeated until there are no more values to compare.

At this point the biggest value is at the end of the list and can be considered sorted (in the image is the number `8`). The `i` pointer is reset to the beginning of the list and the same process starts again. This time the traversing does not touch the elements already sorted and places the next bigger element (the `6` in the image) at the end of the unsorted list to the left of the sorted part. And now both `8` and `6` are considered sorted.

The same procedure is repeated until there are no more swaps during the traverse of the list. At that moment the list is sorted completely (all elements are smaller than the one at their right).

And this can be implemented also quite easily:

```
def bubble_sort(array):
    # Prepare a flag variable that will stop the main loop if nothing has been done in that
cycle
    # That means the array is already ordered and the function can stop
    # The flag starts as True, so execution will enter the loop
    swapped = True
    # Prepare a pointer that will mark the limit between the unsorted and
    # sorted parts of the list.
    j = len(array)
    # Main loop
    while swapped:
        print('cycle')
        # Reset flag at the start of each cycle
        swapped = False
        # Traverse list from the start.
        # Elements already ordered (from j forwards) are not checked
        for i in range(0, j-1):
            # Swap if the current pair (i, i+1) is not sorted
            if array[i] > array[i+1]:
                array[i], array[i+1] = array[i+1], array[i]
                # And rise up the variable flag
                swapped = True
        # In every cycle, one element is added to the sorted part of the list
        j -= 1
```

The algorithm can still be optimized by remembering where the last swap was and updating the sorted pointer (`j`) to that value.

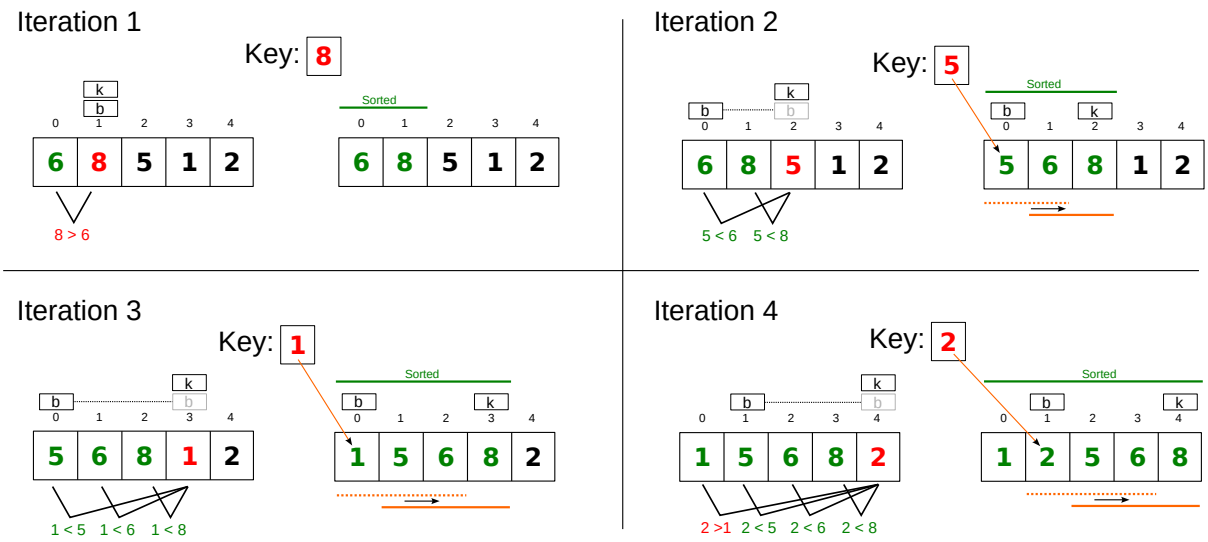
As we can understand from the algorithm, in the worst case (an inverse sorted list), the list is traversed n times, one for each element to be positioned at the right place. And on each cycle, a decreasing number of elements is checked. Something like: $n \cdot (n + (n-1) + (n-2) + ...)$. So at least we can easily say that this algorithm is of time complexity $O(n^2)$ for the worst case.

In the best case it would be $O(n)$, but the average is again $O(n^2)$. Given the number of checks and swaps this algorithm makes, it is usually considered a very bad algorithm and rarely used in practice. Even other algorithms with same time complexity run faster than Bubble sort.

More on this algorithm: Wikipedia entry for Bubble sort

Insertion sort

Insertion sort algorithm starts considering the first element as sorted. Then it iterates the list starting from the second element and on each cycle, it will find the right place for the current element (the key value of that cycle) in the sorted part of the list. It will also move the necessary elements from the sorted part of the list one position to the right, to make room for the key value, and storing the value on that position.



In the image we see the main pointer `k` iterating forwards starting at the second position. This pointer marks the key value at that iteration. At each step of `k`, the pointer `b` iterates backwards from `k` comparing values of pair `b-1` and key until the right place for the key is found. Then, if necessary, it makes room for the key, moving the elements one place to the right and storing the key in the correct place.

In the first iteration nothing needs to be done, as 8 is bigger than 6, but in the second iteration, the right place for the 5 is found to be the first position and both 6 and 8 need to be moved to the right to make room for the 5. And exactly the same happens in the third iteration. And finally in the fourth iteration, the number 2 finds its place at the second position and only some values (5, 6 and 8) need to be moved to the right.

Exercise!

Given the information you have received about the Insertion sort algorithm, try to implement it. Given an array like: `[6, 8, 5, 1, 2]`, the sorted array would be: `[1, 2, 5, 6, 8]`

Don't continue until you have finished the exercise!

A possible implementation could be:

```
def insertion_sort(array):
    # The array will be traversed with a key_index
    for key_index in range (1, len(array)):
        # Store the value of key
        key = array[key_index]

        # Loop backwards through the sorted part of the list (left of key_index) to find the
correct place for the key value
        back_index = key_index
        while back_index > 0 and array[back_index-1] > key:
                back_index -= 1

        # If back_index moved, then some elements have to be reordered
        if back_index < key_index:
            # Move elements to the right
            array[back_index+1:key_index+1] = array[back_index:key_index]
            # And insert the key at the position marked by the back_index
            array[back_index] = key
```

Again and for the same reasons as with Bubble sort, at worst (array inverse sorted) this algorithm is of time complexity $O(n^2)$ and best case is $O(n)$ (array already sorted). Average is also $O(n^2)$. But this is more efficient that Bubble sort, as it makes less element moves than Bubble sort.

For example, the next block of code gives me a runtime of 7 seconds for Bubble sort and a runtime of 3 seconds for Insertion sort on the same array.

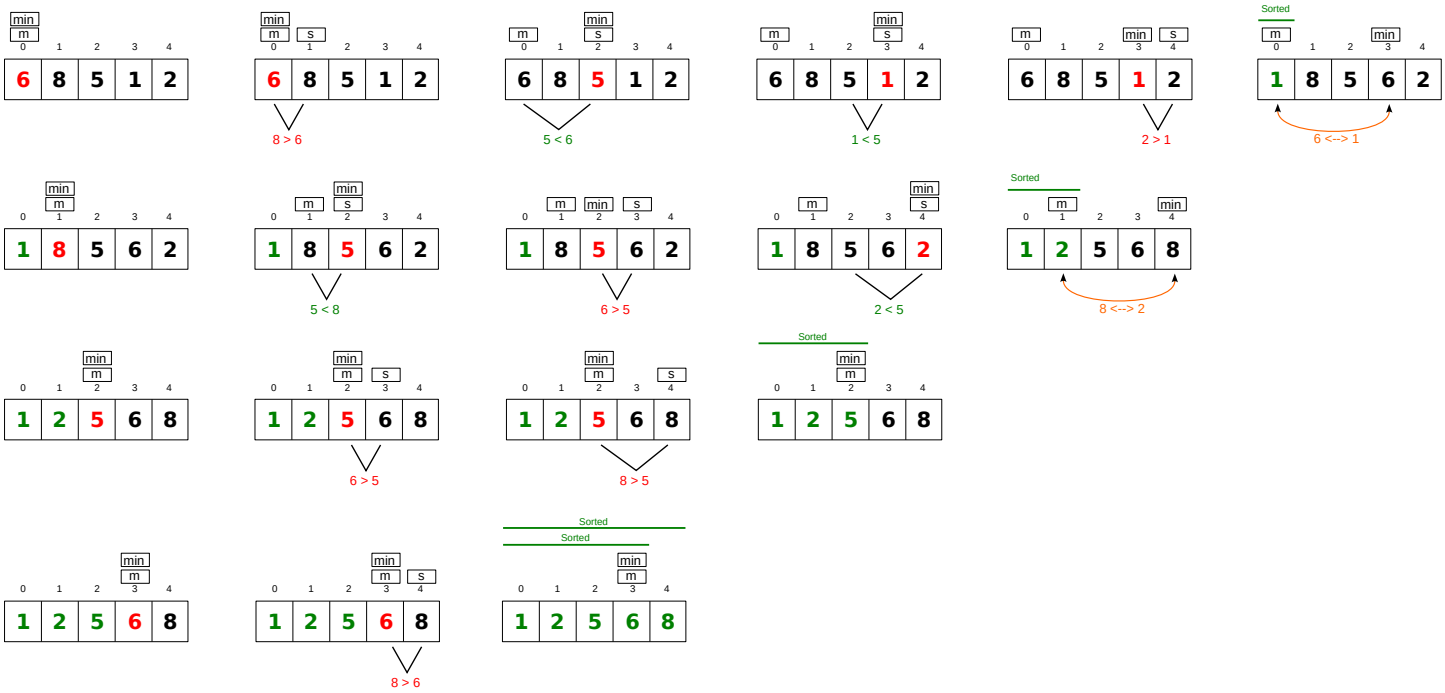
```
array=[randint(0,10000) for _ in range(10000)]

print('Running Bubble sort')
start_time = time.time()
bubble_sort(array.copy())
runtime_bubble = time.time() - start_time

print('Running Insertion sort')
start_time = time.time()
insertion_sort(array.copy())
runtime_insertion = time.time() - start_time

print(f'Bubble sort runtime: {runtime_bubble}')
print(f'Insertion sort runtime: {runtime_insertion}')
```

This kind of algorithm is efficient enough for short arrays and its used abundantly by many software developers when sorting small datasets also due to its simplicity.

More on this algorithm: Wikipedia entry for Insertion sort

Selection sort

Another sorting algorithm with $O(n^2)$ time complexity is Selection sort. This is a very simple algorithm with a very simple idea, iterate repeatedly the array finding the minimum value and swap it with the next unsorted position starting from the left.




```python
def selection_sort(array):
    # Traverse the array
    for main_index in range(0, len(array)-1):
        # On each cycle, minimum index is set to current index
        min_index = main_index
        # Search the rest of the array for a value smaller than
        # the one at min_index. If found, update min_index to point to it
        for search_index in range(main_index+1, len(array)):
            if array[search_index] < array[min_index]:
                min_index = search_index
        # If the index of the minimum value is different than the index
        # of the current value, then swap them.
        if min_index != main_index:
            array[min_index], array[main_index] = array[main_index], array[min_index]
```

As previous algorithms, this one is of time complexity $O(n^2)$ in every case (worst, average and best cases). And being also quadratic it has the same advantage with small datasets. Other than that, it is not very efficient, and it lies near Insertion sort in performance. Usually Insertion sort is preferred to

this algorithm, but Selection sort can have the advantage of making less writes, that can be better for some write limited devices.

More on this algorithm: Wikipedia entry for Selection sort

Quicksort

Quicksort uses a different strategy than the algorithms we have seen. It belongs to the "Divide-and-conquer" type of algorithms. This type of algorithms tries to divide the main task in simpler subtasks, usually through recursivity. In this case Quicksort selects a pivot element and divide the elements of the array in two groups: elements smaller than the pivot and elements bigger than the pivot. And then it applies the same algorithm to each of the groups. One interesting property of this method is that once the pivot is swapped, it lies in the correct position it should be in the sorted array. Even if the rest of elements are not yet in their right place.

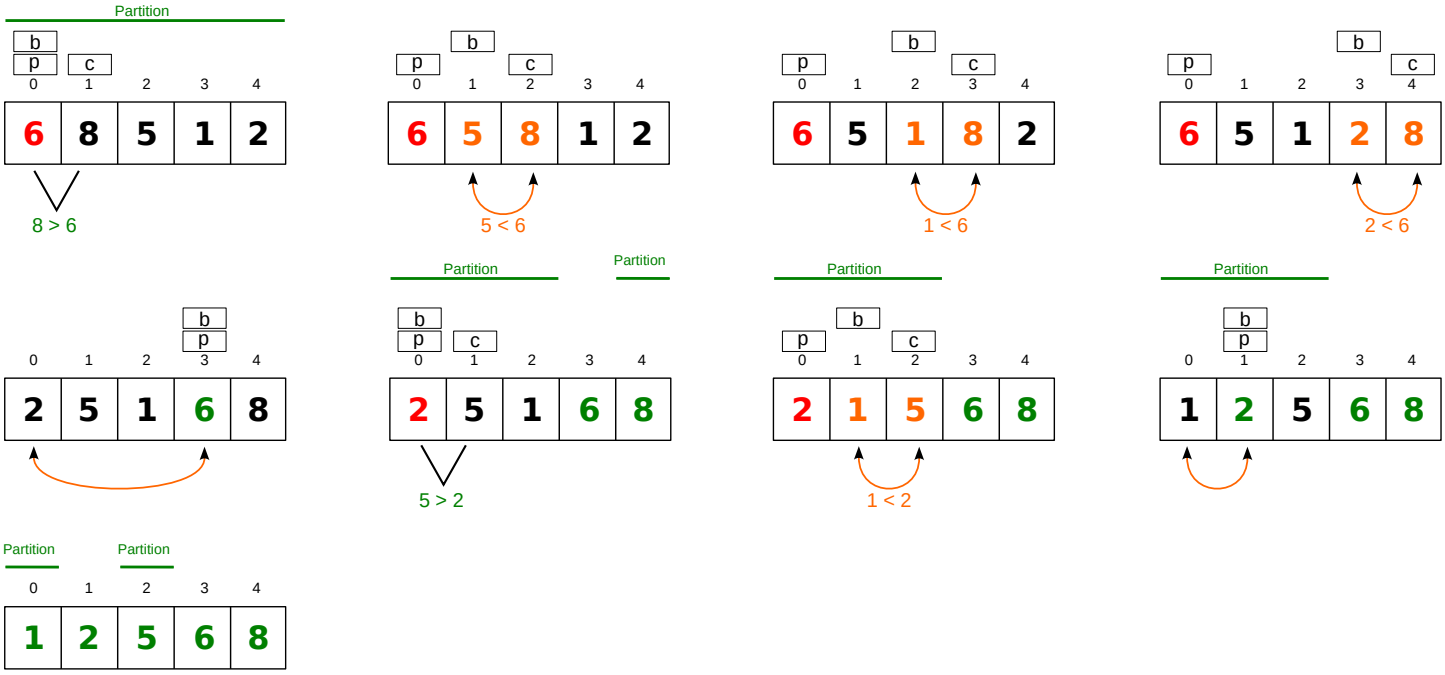
The pivot selection has a great influence in the overall performance, but this is a whole subject in its own and can be quite complicate. For simplicity we are going to use the first element of the partition. The partition is the part of the array being sorted in the current call. The first time the function is called, the partition is the whole array, but in subsequent calls it's just a smaller part of the array. Other popular choice for the pivot is the last element of the partition.

important notice

Selecting the first element of the partition as the pivot has the effect of causing a worst-case scenario (from performance view) when the array is already sorted, that can be on the other hand a quite common case. So, it is better to select other pivot.

In any case we are going to use it as the pivot just for simplicity. But take in mind that it probably should be avoided in real life. Selecting the last element as the pivot is not very difficult, but it is also not a good option in some cases (but it is still better than the first element).

Once the pivot is selected, elements smaller or equal than the pivot are placed to the right of the partition and elements bigger than the pivot are placed to the left of the partition. When all elements are replaced, the pivot is switched with the last of the smaller values and the same process is applied to each of the partitions that were form (the elements to the right of the pivot, and the elements to the left of the pivot), usually recursively.



In the image we can see that the whole array is the partition, and the pivot is marked with a `p` index. The `b` index marks the border (the last element) of the smaller elements, being at first the pivot itself. The `c` index marks the element currently being checked against the pivot. As we can see, the algorithm iterates through the partition checking each element and if the element is smaller than the pivot, it is swapped with the first element of the bigger elements, that is at position `b+1`. After each swap `b` index is incremented.

In the first step 8 is bigger than 6 and it is not added to the smaller part. In the second step, 5 is smaller than the pivot (5 < 6) and therefore it is added to the smaller elements by swapping it with the first of the bigger elements (that is in position 1 at this moment), and then incrementing `b`. Same happens in steps 3 and 4. As all elements have been checked against the pivot, the pivot itself is swapped with the last of the smaller elements, creating two partitions: one to the left of the pivot (position 0-2) and one to the right of the pivot (position 4).

As the right partition is only one element, it does not need to be sorted, but algorithm is called with left partition as current partition. The algorithm is applied to the left partition the same way as before and if necessary, it would be called on smaller partitions, but being this a very small dataset, the algorithm finishes without more calls on partitions.

There are some special cases to be taken care of, like when the selected pivot happens to be the smaller element or the bigger element of the partition. Or when the element being added to the left (smaller elements) is already in the position it should be. In that case there is no need to swap elements, just update the border index.

A possible implementation could be like this:

```python
def quick_sort(array, left_index=None, right_index=None):
    # Set these values so the function can be called by the user with only the array as parameter
    if not left_index:
        left_index = 0
    if not right_index:
        right_index = len(array) - 1
    # The pivot selection is a problem in its own.
    # For now let's use the first element, but any other element could have been chosen (with necessary algorithm's adjustments)
    pivot_index = left_index
    # We set a border index to indicate that values less or equal than pivot are to
    # its left (border index inclusive) and values bigger than pivot are at its right
    # Right now the only element less or equal than the pivot, is the pivot itself.
    border_index = left_index
    # And start traversing the partition
    for current in range(left_index+1, right_index+1):
        # if the value of the current position is less than pivot,
        # let's add it (current value) to the minors part
        if array[current] <= array[pivot_index]:
            # Update border as there is one minor more now.
            border_index += 1
            # Check if we actually need to do the swap. If current index was
            # the next value after minors, then no swap is necessary
            if current > border_index:
                # If that was not the case, then swap their values
                array[current], array[border_index] = array[border_index], array[current]
    # After traversing the partition, the pivot can be swapped with the last of the minors.
    # If pivot is the only  minor, then there is no need to swap.
    if border_index != pivot_index:
        # swap
        array[border_index], array[pivot_index] = array[pivot_index], array[border_index]
        pivot_index = border_index  # Update pivot index after swap
    # The pivot is in its position and all values less or equal than pivot are at the left,
    # and all values bigger than pivot are at the right.
    # Now call recursively this function on both partitions.
    # Check if left partition has at least 2 elements
    if (pivot_index - left_index) > 1:
        quick_sort(array, left_index, pivot_index-1)
    # Check if right partition has at least 2 elements
    if (right_index - pivot_index) > 1:
        quick_sort(array, pivot_index+1, right_index)
```

Quicksort algorithm is quite efficient, but it's not so easy to analyze like other sorting algorithms. This algorithm has different time complexities depending on the case. The worst case is with unbalanced arrays/partitions. If, for some reason, the pivot happens always to be the first or the last element of

the partition then there are going to be n iterations where in each one of them will be $n-1$, $n-2$, $n-3$, etc... That is $\sum_{i=0}^{n}(n-i)$. From math theory, we know that $\sum_{i=1}^{n} n$ (the same series but in the opposite order) is $\frac{n\cdot(n+1)}{2}$. So that means, that we can resolves our series as $\frac{(n-1)\cdot n}{2}$, and that function have a leading polynomial of n^2. So the worst case is $O(n^2)$.

On the other hand, the best case is when the array/partition is balanced and the subpartitions are of the same size. This is like a binary tree. In a binary tree, the height of the tree was $\log n$, and in the same way, the maximum number of recursive layers of calls is $\log n$. In each one of these layers and joining all the partitions on that layer, we have a time complexity of $\Omega(n)$, so overall, the time complexity for the whole algorithm in the best case is $\Omega(n \cdot \log n)$.

This is much better than previous algorithms but is only for the best case. The average case is much difficult to calculate, and actually only estimations can be given, but the general consensus is that is closer to the best case than otherwise, so we are still talking about a quite efficient algorithm.

An empiric example could be done with the following block of code that continues the tests we did before:

```
start_time = time.time()
quick_sort(array.copy())
runtime = time.time() - start_time
print(f'Quicksort runtime: {runtime}')
```

In the same computer as before, this gives a runtime of 0.028 seconds. This is 1/100 of the time that Insertion sort needed.

The only downsize of this algorithm compared to the ones we saw before, is that it can possibly use more memory space to complete the task. But other than that, is one of the best algorithms available. There are still others that can be better, but some tradeoffs have to be done, maybe.

More on this algorithm: Wikipedia entry for Quicksort

Heapsort

The idea behind Heapsort algorithm is very simple, as a heap structure is quite efficient and it returns elements in order, let's use it as a helper structure to solve the sorting problem. And implementing this simple idea is also very simple, create the heap, insert all values on the heap and then pop all values back from the heap. It can be translated into code like this:

```python
def heap_sort(array):
    # Create the heap
    h = Heap()
    # Insert the values on the heap
    for val in array:
        h.insert(val)

    # Return a sorted array, created by just popping the heap
    return [h.pop() for _ in range(len(array))]
```

This is already quite efficient. This algorithm pushes all elements into the heap and then pops all elements from it. That would be $n + n$ (or $2 \cdot n$) operations. And each push and pop operations to and from the heap are of time complexity $O(\log n)$. So overall, the algorithm is of time complexity $O(n \cdot \log n)$.

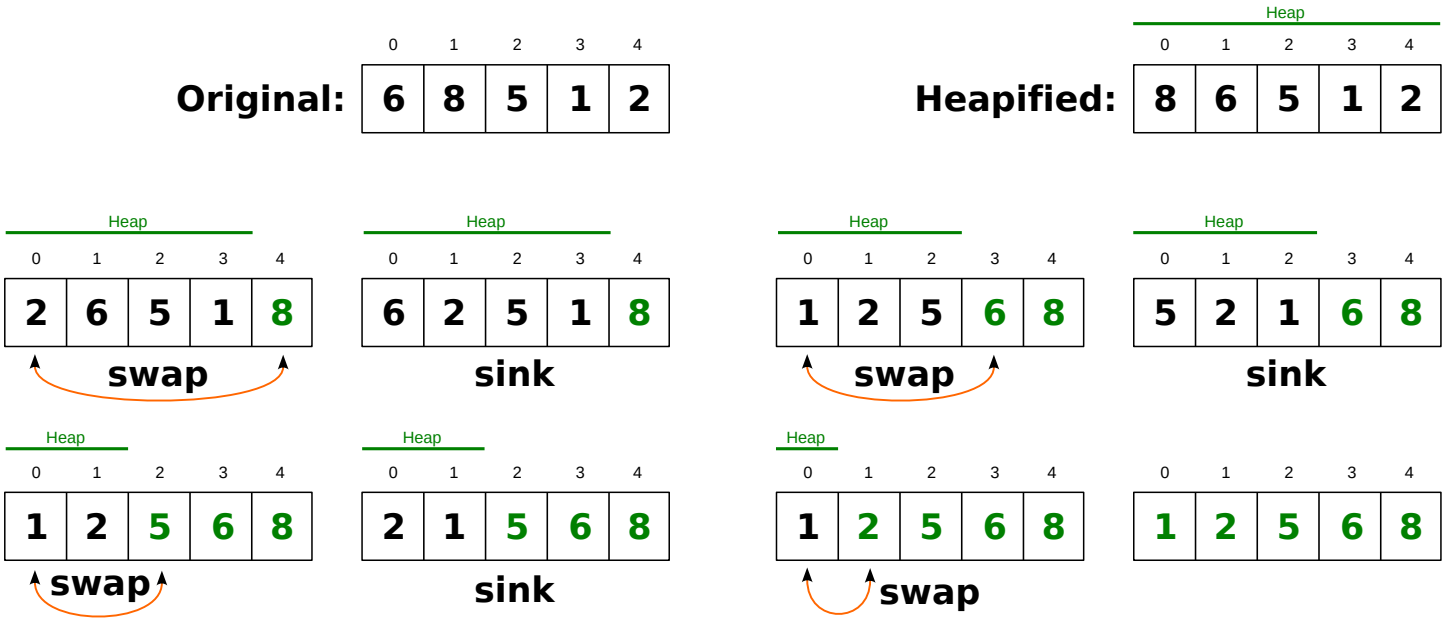
A quick test, like before, shows a good runtime for this algorithm:

```python
start_time = time.time()
_ = heap_sort(array.copy())
runtime = time.time() - start_time
print(f'Heapsort runtime: {runtime}')
```

The runtime in the same computer as before is 0.12 seconds. A little bit more than Quicksort but far less than Bubble sort or insertion sort.

Anyway, the actual Heapsort algorithm is a little more complex. The problem with the implementation showed here (commonly called the naive implementation) is that it uses a completely different structure to do the sorting, so we need the same amount of extra free space than the array itself. In other words, its space complexity is $O(n)$. The actual Heapsort algorithm uses the array itself to store the heap and to do the actual sorting. It also uses a Max heap to simplify things. So now the algorithm should do something like:

1. Convert the array in a Max heap (aka "Heapify")
2. Use a variable to hold the length of the heap. At first is the same as the length of the array
3. As the array is a max heap, the max value is at the root (position 0), so swap positions 0 and last position of the heap
4. Decrement the length of the heap and sink the value at the root of the heap to maintain heap property
5. Repeat until length of the heap is 0.



As we see, the swap and sink part is quite easy. The trickiest part of the algorithm is to convert the array in a heap in-place, that is, without using an external structure or memory space. For that we can use a helper function that sinks the value of a node.

The helper function **sift_down** accepts three parameters: the array that contains the heap, the index of the node we want to sink and the index of the last node of the heap. During the process we saw before, we can notice that the sorted elements are stored at the right of the array, so at some point we want to limit the length of the heap, as the sorted values are not part of the heap.

The function consists on a loop that will run while there is at least a left child (of the current node) to compare values with. Inside the loop it will store the **index** of the current node in a temporary variable and then it will check if the left child's value is bigger than the value pointed by the temporary variable (that is right now pointing to the current node). If left child is bigger, it will update the value of the temporary variable with the index of the left child.

Then it will do the same with the right child, but checking first if it exists. If its value is bigger than the node pointed by the temporary value (that could be the current node or the left child), update the temporary value with the index of the right child.

After that the temporary value points to the biggest of the three nodes and if it is different than the current node, it's just a matter of swapping their values. On the other hand, if the current node is bigger than its children, the function can return.

And that is basically the sift_down function. The process will be repeated until there are no more children (a leaf node) or the node can't be sunk more due to be bigger than its children.

Exercise!

Given the information you have received about the sift_down function, implement it
Given an array like: `[6, 1, 5, 8, 2]`, and calling the sift_down function with the parameters `array, start=1, end=4` (sink the node with the value of 5), the array should be modified to look like this: `[6, 8, 5, 1, 2]`

Don't continue until you have finished the exercise!

```python
def sift_down(array, start, end):
    """
    This function sinks (if necessary) the given node of a MaxHeap structure

    Parameters:
    - array: The heap array
    - start: The index of the node that should be sunk.
    - end: The end of the heap inside the array. The index of the last node

    Returns: None
    """
    # Let's use a better variable name inside the function
    current = start

    # Loop while there is a left child to compare with
    while (left_child := 2*current+1) <= end:
        # Initialize swap var with current
        swap_index = current

        # If left child is smaller than current swap index, use it as swap index
        if array[swap_index] < array[left_child]:
            swap_index = left_child

        # If there is a right child and it is smaller than current swap index, use it as
swap index
        right_child = 2*current+2
        if right_child <= end and array[swap_index] < array[right_child]:
            swap_index = right_child

        # If swap index has not changed, current element is bigger than its children and
function can return
        if swap_index == current:
            return

        # Swap values at current and swap index
        array[current], array[swap_index] = array[swap_index], array[current]
        # Update current to be swap_index (left or right child)
        current = swap_index
```

It accepts as parameters the whole array, the index of the node to be sunk (marked as `start`) and the last element of the heap. This way we can exclude elements from the array.

This function is going to be used on two places: When converting the array and later when doing the swap-sink process. To convert the array in a Max heap, we start considering the array as a heap that is not at this moment following the heap properties, so we would need to sink all nodes from the bottom to the top to be sure that the structure is a correct heap. However, a heap is also a complete

binary tree. That means that all nodes of a layer must be filled before continuing to the next one. And from this property can be inferred that the last half of the nodes are actually leaves (actually it's $\frac{n}{2}$ or $\frac{n}{2} + 1$ of the nodes). Leaves don't need to be sunk, so with that information we can calculate that we could start sinking all nodes from `(size//2)-1` (one node before the half) backwards.

Original:

0	1	2	3	4
6	8	5	1	2

0	1	2	3	4
6	8	5	1	2

Half of the elements are leaves

0	1	2	3	4
6	8	5	2	1

Sink number 8 at position 1

0	1	2	3	4
8	6	5	2	1

Sink number 6 at position 0

That would convert the array into a Max heap. After that, it's just a matter of using the procedure given before.

Exercise!

Given the information you have received about the Heapsort algorithm, implement it using the helper function we have just saw.
Given an array like: `[6, 8, 5, 1, 2]`, the sorted array would be: `[1, 2, 5, 6, 8]`

Don't continue until you have finished the exercise!

A possible implementation could be:

```
def heap_sort(array):
    # Heapify the array with a Max heap
    for start in range(len(array)//2-1, -1, -1):
        sift_down(array, start, len(array)-1)

    # As using a max heap, the heap will return the max values,
    # so they should be placed at the end of the array
    end = len(array)-1  # Last position

    # Loop while heap is not empty
    while end > 0:
        # Swap root and end places
        array[0], array[end] = array[end], array[0]
        # Decrease end. Now heap is one element shorter
        end -= 1
        # Sink the value at the root to maintain the heap property
        sift_down(array, 0, end)
```

This version of the heapsort algorithm is, same as before, of time complexity $O(n \cdot \log n)$. And the test block code that we had before shows a runtime of 0.004 seconds. So, this version is faster than the previous one. And it is logical as this only need to sink half of the elements and doesn't need to do as much copy operations as in the previous one.

More on this algorithm: Wikipedia entry for Heapsort

Other sorting algorithms

There are more sorting algorithms. It is actually a well-studied area of programming and many algorithms and theories have been developed on it.
Some of the algorithms you can study by yourself are: Timsort, Introsort or Cubesort